

NeuroMemoryStudy - April 29, 2026 Dev Journal

Objective

The primary objective of today's session was to troubleshoot the backend Python data acquisition engine (engine.py) and begin integrating the "Project Architecture"—specifically, a real-time visualization and FFT analysis pipeline over a WebSocket bridge between the Python backend and the C# WPF frontend.

Progress & Achievements

1. Diagnosing Data Acquisition (LSL)

Problem: The Python engine would hang indefinitely at the "Acquisition loop started. Buffering raw data..." message despite the Muse headset being connected via BlueMuse. **Diagnosis:** The engine was secretly dropping all the data due to strict artifact rejection. The Peak-to-Peak (ptp) threshold was set at a strict 100uV. Because forehead muscles and eye movements generate large electrical spikes (often > 150uV), even small blinks were triggering the "dirty" flag. The loop simply ignored dirty chunks, making it seem like no data was arriving.

2. Refining the Artifact & DSP Pipeline

To balance strictness (for research) with practical usability:

- **Narrowed the Bandpass Filter:** Changed the high-cut from 40Hz to 30Hz (1-30Hz). This effectively strips out the high-frequency muscle noise (EMG) without losing target brainwaves (Alpha/Beta).
- **Threshold Adjustment:** Bumped the artifact gating threshold from 100uV up to 150uV to allow for micro-movements.
- **Hold Mechanism:** Added a state tracker (last_valid_metrics). If a momentary artifact occurs (like a blink), the engine holds the previous good state instead of failing entirely. This will keep the neurofeedback audio smooth.

3. Real-Time Prototyping

- **Matplotlib Prototype:** Implemented a quick, temporary UI in Python using matplotlib to verify the DSP pipeline. We proved that the 1-30Hz filtering worked. However, it confirmed that Python's matplotlib is too slow for real-time looping (it began to lag after ~5 seconds).

4. Moving to Official Project Architecture (WebSocket Bridge & WPF)

We transitioned to the intended C# & Python architecture:

- **Python Engine Rewrite:**
 - Removed matplotlib.
 - Added websockets and asyncio to create a lightweight WebSocket server broadcasting on port 8765.
 - Implemented Fast Fourier Transform (FFT) logic using scipy.signal.welch on the filtered AF7 channel.

- Extracted the specific Alpha (8-12Hz) and Beta (13-30Hz) bands, calculated their powers via NumPy (`p.trapz`), and generated JSON payloads.
- **C# WPF Frontend (MainWindow.xaml):**
 - Installed ScottPlot.WPF and standard JSON/WebSocket client libraries.
 - Built the UI with labeled axes.
 - Added two real-time charts:
 1. **EEG Time Series:** Shows the Raw (gray) vs Filtered (blue) AF7 signal.
 2. **FFT Power Spectral Density:** Displays the frequency distribution up to 40Hz with vertical color bands highlighting the Alpha and Beta ranges.
 - Added UI status text tracking connection state, Alpha/Beta power, and the Alpha/Beta Ratio for live neurofeedback tracking.

Next Steps

1. **Test the Bridge:** Boot up BlueMuse, run the Python Engine, and then run the C# UI to verify the charts are performant and correctly interpreting the JSON stream.
2. **Audio Engine Implementation:** Begin writing the audio feedback loop in Python or C# to dynamically modulate the nature soundscape based on the calculated Alpha/Beta ratio.